

Dossier de projet ThermoSense

Section A — Contrat API et décisions de design

Ressources définies

L'API ThermoSense pilote un réseau de capteurs thermiques et d'actionneurs répartis dans des bâtiments. Le modèle de données suit une hiérarchie stricte :

Building → **Zone** → **Sensor / Actuator / AlertThreshold**

- **Building** : racine de la hiérarchie, représente un bâtiment physique.
- **Zone** : subdivision d'un bâtiment, pivot de la gestion des droits (un opérateur est lié à une zone).
- **Sensor** : capteur attaché à une zone, produit des mesures (température, humidité...).
- **Measurement** : relevé horodaté produit par un capteur (valeur, type, timestamp).
- **Actuator** : appareil actionnable (chauffage, ventilation, climatisation) attaché à une zone.
- **ActuatorCommand** : commande envoyée à un actionneur, avec un statut traçable (PENDING / SENT / DONE).
- **AlertThreshold** : seuil d'alerte configuré par zone, définissant des valeurs min/max pour un type de mesure.
- **User** : utilisateur de l'API, avec un rôle (ADMIN, OPERATEUR, LECTEUR, DEVICE_IOT) et un lien optionnel à une zone.

Évolutions principales du contrat

Le contrat a évolué de façon significative depuis sa version initiale. La version initiale couvrait uniquement les ressources métier de base (Building, Zone, Sensor, Measurement, Actuator, ActuatorCommand, AlertThreshold) avec leurs endpoints CRUD fondamentaux, sans mécanisme d'authentification ni de contrôle d'accès.

Les évolutions majeures introduites au fil du module sont les suivantes :

Authentification et contrôle d'accès (RBAC) : ajout d'un système complet d'authentification JWT avec quatre rôles distincts (ADMIN, OPERATEUR, LECTEUR, DEVICE_IOT). Chaque rôle dispose de droits précis par ressource, et le rôle OPERATEUR est de plus limité à sa propre zone via un claim **zoneId** porté dans le token. Ce système couvre l'ensemble des endpoints via un middleware dédié.

Pagination et filtrage : les endpoints de liste exposant potentiellement de grands volumes de données (notamment **GET /measurement**) ont été enrichis de paramètres de filtrage (**startDate**, **endDate**, **type**, **zoneId**, **sensorId**) et de pagination (**limit**, **offset**), afin de maîtriser la consommation réseau dans un contexte mobile et IoT.

Rate limiting : trois limiteurs distincts ont été introduits — un sur l'endpoint de login (8 échecs / 10 min / IP), un sur le refresh token, et un sur les écritures critiques (60 req / 15 min / IP). Ce mécanisme protège à

la fois contre les attaques par force brute et contre la saturation des actionneurs.

Idempotence et mode dégradé : un mécanisme d'idempotency key (header ou body) a été ajouté sur les opérations à effet de bord, avec un cache TTL 24h. Un mode dégradé a également été introduit pour les mesures : si la base de données est inaccessible, la mesure est mise en file d'attente en mémoire et persistée dès le rétablissement, avec retour d'un **202 Accepted** au lieu d'un **500**.

Stratégie de versioning

Suite aux audits croisés qui ont identifié l'absence de versioning comme une non-conformité, un versioning par préfixe d'URL a été introduit dans le contrat. La stratégie retenue est un préfixe **/v1/** sur l'ensemble des endpoints, accompagné d'une politique de dépréciation explicite : maintien de l'ancienne version pendant une période définie, header **Deprecation** sur les réponses des endpoints obsolètes, et documentation des breaking changes dans le changelog du dépôt.

Ce correctif a été appliqué après l'audit croisé, ce qui explique son absence dans la version initiale du contrat (S1).

Décisions de design difficiles

Décision 1 — Piloter les actionneurs via une sous-ressource **ActuatorCommand**

Problème posé : dans un contexte mobile avec réseau instable, une requête de commande peut être émise plusieurs fois. Sans traçabilité, une double activation d'un chauffage ou d'une vanne peut avoir des conséquences physiques réelles.

Options envisagées : un simple **PATCH /actuators/{id}** avec le nouvel état souhaité. Cette approche est plus simple mais ne permet pas de distinguer une commande légitime d'un doublon réseau.

Choix retenu : création d'une ressource **ActuatorCommand** distincte, avec un statut traçable (PENDING / SENT / DONE) et un support de l'idempotency key. L'application peut interroger l'état de la commande sans risquer de la rejouer.

Compromis assumé : légère complexité supplémentaire côté client, qui doit gérer deux appels (création de commande + consultation du statut) au lieu d'un seul. Ce compromis est acceptable au regard de la criticité des actionneurs physiques.

Décision 2 — Exposer l'état synthétique dans la ressource **Zone**

Problème posé : l'application mobile doit afficher rapidement l'état global d'une zone (température actuelle, état du chauffage) sans multiplier les requêtes réseau, coûteuses sur un réseau mobile dégradé.

Options envisagées : un endpoint dédié **GET /zones/{id}/summary**, qui oblige le client à faire un appel supplémentaire par zone affichée.

Choix retenu : intégration de champs synthétiques (**current_temperature**, **heating_status**) directement dans la réponse **GET /zones/{id}**, évitant un aller-retour réseau supplémentaire.

Compromis assumé : violation partielle du principe REST de séparation des ressources — des données appartenant techniquement aux capteurs sont exposées au niveau de la Zone. Ce serait un mauvais choix si le calcul de l'état synthétique était coûteux côté serveur.

Décision 3 — URL courtes avec imbrication limitée

Problème posé : une hiérarchie complète dans les URLs (/buildings/{id}/zones/{id}/sensors/{id}) est lisible mais verbeuse et rigide à faire évoluer.

Options envisagées : hiérarchie complète dans toutes les URLs.

Choix retenu : structure mixte — imbrication uniquement lors de la création (ex : **POST** /building/{id}/zone, **POST** /zone/{id}/sensor) pour ancrer la relation métier, mais accès direct par ID pour les opérations sur une ressource connue (ex : **GET** /sensor/{id}). Ce choix est valide car les IDs sont globalement uniques.

Compromis assumé : perte du contexte hiérarchique dans les URLs de lecture directe. Ce serait un mauvais choix si les IDs n'étaient pas uniques globalement mais seulement dans leur contexte parent.

Section B — Dossier de sécurité

Synthèse du threat model

L'analyse de menaces a porté sur quatre endpoints critiques, sélectionnés pour leur impact métier ou leur exposition au risque :

- **POST** /actuators/{id}/commands — pilotage d'actionneurs physiques
- **PUT/PATCH** /zones/{id}/thresholds/{id} — modification des règles d'alerte
- **GET** /sensors/{id}/measurements — extraction de données de télémétrie
- **POST** /buildings — création de la racine de la hiérarchie

Frontières de confiance identifiées :

- Entre le client mobile et l'API (réseau non maîtrisé, token stocké sur le device)
- Entre l'API et la base de données (accès local, supposé maîtrisé)
- Entre l'API et les objets connectés (DEVICE_IOT, authentification JWT dédiée)

Synthèse de l'audit

Deux audits croisés ont été conduits par des groupes tiers. Les non-conformités identifiées sont les suivantes :

Criticité	Non-conformité	Statut
-----------	----------------	--------

Bloquant	Idempotence insuffisamment documentée sur les opérations à effet de bord	Implémentée, documentation à renforcer
Bloquant	Absence de stratégie de versioning	Corrigé après audit — versioning /v1/ introduit
Majeur	Rate limiting absent sur certains endpoints d'écriture	Partiellement corrigé
Majeur	Erreurs non structurées selon RFC 7807	Risque résiduel assumé
Mineur	Exemples de requêtes/réponses peu réalistes	Risque résiduel assumé
Mineur	AuthN/AuthZ non documentée dans le contrat OpenAPI	Risque résiduel assumé

Preuves de tests

Tests d'authentification (AuthN) :

- T1 : génération d'un token JWT valide via **POST /auth/login** → 200 OK avec access token et refresh token ✓
- T2 : vérification des claims (sub, role, scope, exp, aud) dans le token décodé ✓
- T3 : requête sans token → 401 ; requête avec token valide → 200 ✓
- T4 : token expiré → 401 ✓ ; signature invalide → 401 ✓ ; audience incorrecte → 401 ✓

Tests d'autorisation (AuthZ — BOLA/BFLA) :

- A1 : opérateur zone A accède à un capteur de sa zone → 200 ✓
- A2 : opérateur zone A tente d'accéder à un capteur zone B → 403 (BOLA) ✓
- A3 : lecteur tente un DELETE → 403 avec raison **role_insuffisant** (BFLA) ✓
- A4 : admin effectue un DELETE → 204 ✓
- A5 : device IoT poste une mesure sur son propre capteur → 201 ✓
- A6 : device IoT tente de poster sur un capteur étranger → 403 (BOLA) ✓

Tests de rate limiting :

- R1 : 3 requêtes sous le seuil → 201 avec headers X-RateLimit ✓
- R2 : rafale de 15 requêtes → 429 avec Retry-After dès la 11e ✓
- R3 : après 60s, compteur réinitialisé → 201 ✓

Observabilité :

- O1 : accès refusé (403) → log JSON **access_denied** avec correlation_id ✓
- O2 : rate limit (429) → log JSON **rate_limited** ✓
- O3 : aucune donnée sensible (token, password) dans les logs ✓
- O4 : Correlation ID traçable de bout en bout ✓

Risques résiduels assumés

Risque 1 — Idempotency key en mémoire. Le cache d'idempotence est stocké en mémoire avec un TTL de 24h et un nettoyage automatique toutes les heures. Un redémarrage du serveur efface le cache, ce qui peut permettre la réexécution d'une commande si le client renvoie la même clé après un crash. Ce risque est assumé ; en production, le cache serait externalisé sur Redis.

Risque 2 — File de mode dégradé volatile. Les mesures mises en file d'attente lors d'une indisponibilité de la base sont stockées en mémoire. Un crash serveur pendant la panne entraîne la perte définitive de ces mesures. Ce risque est assumé et documenté ; en production, la file serait persistée (Redis ou stockage disque).

Risque 3 — Erreurs non conformes RFC 7807. Le format des réponses d'erreur n'est pas entièrement harmonisé. Certaines réponses retournent un message libre plutôt qu'un objet structuré. Ce risque est assumé délibérément : la priorité a été donnée à l'implémentation des mécanismes de sécurité (RBAC, rate limiting) et de résilience (idempotence, mode dégradé), dont l'impact opérationnel est supérieur à la normalisation du format d'erreur. La standardisation RFC 7807 serait prioritaire dans une phase de stabilisation du contrat avant mise en production.

Section E — Bilan d'architecture et recommandations

Schéma d'architecture final

(Voir document joint : architecture-thermalsense.pdf)

L'architecture repose sur un backend Express.js/TypeScript exposant une API REST documentée via Swagger (OpenAPI 3.0). Chaque requête traverse successivement : l'injection d'un X-Request-ID, le rate limiter, la vérification JWT, le contrôle RBAC, puis le contrôleur métier. La persistance est assurée par SQLite via Prisma. Deux mécanismes transversaux complètent l'architecture : un store d'idempotence en mémoire (TTL 24h) et une file de mode dégradé pour les mesures (flush toutes les 5s, 5 tentatives max).

Composants principaux :

- Clients : capteurs IoT (DEVICE_IOT), opérateurs/admins (navigateur ou API client), lecteurs
- Sécurité : authLoginRateLimiter, authRefreshRateLimiter, criticalWriteRateLimiter, JWT middleware, RBAC middleware, securityLogger
- Contrôleurs : building, zone, sensor, actuator, actuatorCommand, alertThreshold, auth, measurement
- Utilitaires : idempotencyStore, measurementQueue, requestContext, swagger
- Persistance : SQLite (Prisma) — entités Building, Zone, Sensor, Actuator, ActuatorCommand, AlertThreshold, User, Measurement

Lien du dépôt et tests

Le code source, le contrat OpenAPI, la suite de tests et le README sont disponibles sur le dépôt Git : <https://github.com/Adrienraynaudd/ThermalSense>

Les tests sont automatisés et exécutables via `npm test` depuis la racine du projet. Ils couvrent les tests fonctionnels des endpoints ainsi que les tests de sécurité (AuthN, AuthZ BOLA/BFLA, rate limiting, observabilité).

3 décisions à prendre différemment si on recommençait

1 — Externaliser le cache d'idempotence et la file de mode dégradé dès le début.

Stocker ces deux mécanismes en mémoire était pragmatique pour démarrer vite, mais crée des limites opérationnelles importantes : toute perte du processus efface à la fois les clés d'idempotence et les mesures en attente. Partir directement sur Redis aurait rendu ces mécanismes production-ready sans complexité excessive, et aurait évité de devoir les qualifier comme risques résiduels en fin de projet.

2 — Structurer les erreurs selon RFC 7807 dès le premier endpoint.

Les erreurs ont été ajoutées au fil du développement, sans schéma commun imposé dès le départ. Le résultat est une inconsistance entre les endpoints (certains retournent un objet `{ error, reason, message }`, d'autres un message libre). Définir un schéma `Error` unique dès la S1 et l'appliquer systématiquement aurait été trivial et aurait évité ce point de dette technique identifié par les deux audits.

3 — Intégrer le versioning dès la S1 plutôt qu'en correction post-audit.

Bien que le versioning ait finalement été introduit suite aux audits croisés, l'avoir intégré dès le début aurait garanti sa cohérence sur l'ensemble du contrat dès la version initiale. Le retro-fitting d'un préfixe `/v1/` en cours de projet nécessite de vérifier la cohérence de tous les endpoints existants, alors qu'une intégration initiale n'aurait coûté qu'une ligne de configuration.

Axes d'amélioration prioritaires pour la production

1 — Versioning et gestion du cycle de vie du contrat. Introduire un préfixe `/v1/`, une politique de dépréciation (header `Deprecation`, délai de migration), et un changelog maintenu.

2 — Externalisation des états en mémoire. Migrer le cache d'idempotence et la file de mode dégradé vers Redis pour garantir la persistance en cas de redémarrage et permettre le déploiement multi-instances.

3 — Notifications temps réel. L'architecture actuelle est exclusivement pull (le client interroge l'API). Pour un usage IoT production, il faudrait ajouter un mécanisme push (webhooks ou MQTT bridge) pour notifier les clients des alertes et des changements d'état sans polling.

4 — Normalisation complète des erreurs. Harmoniser toutes les réponses d'erreur autour d'un schéma RFC 7807 unique, avec des codes métier explicites et une indication de retryability.

5 — Gestion multi-zones pour les opérateurs. Le modèle actuel lie un OPERATEUR à une seule zone (claim `zoneId` dans le JWT). En production, un opérateur peut être responsable de plusieurs zones, ce qui nécessiterait un claim `zoneIds` (tableau) et une adaptation du middleware RBAC.

Section C — Architecture SOA/SOAP

Contexte d'intégration

ThermoSense s'intègre avec deux systèmes externes : une GMAO (Gestion de Maintenance Assistée par Ordinateur) tierce pour la gestion des interventions sur les équipements, et un système de facturation SaaS (Pennylane) pour la synchronisation des données de consommation.

Analyse des besoins d'intégration

Intégration	Contrainte principale	REST suffisant ?	SOAP/SOA pertinent ?	Justification
GMAO (Planon)	Traçabilité bout-en-bout des alertes et ordres d'intervention sous gouvernance SI	Partiellement	Oui	Contrat formel et interop legacy possibles ; SOAP utile si contraintes fortes imposées
Facturation (Pennylane)	Synchronisation fiable des données de consommation vers un SaaS financier	Oui	Non	API moderne JSON, besoins couverts avec idempotence + gestion d'erreurs

Architecture retenue : intégration hybride avec la GMAO

Cas d'intégration retenu : ThermoSense doit déclencher automatiquement des ordres de réparation dans la GMAO tierce dès qu'un capteur franchit un seuil d'alerte critique ou passe au statut « en panne ». La contrainte déclenchante est la criticité opérationnelle : la panne d'un capteur dans une zone sensible (ex : salle serveurs) impose une garantie absolue d'acheminement de la notification sous peine de rupture de SLA.

Choix retenu : architecture hybride — notre API REST/OpenAPI actuelle pour l'application mobile, connectée au service SOAP/WSDL natif du partenaire via un module client SOAP dédié au sein du backend ThermoSense.

Justification par trois critères :

Critère 1 — Fiabilité de l'acheminement (WS-ReliableMessaging). En cas de défaillance thermique majeure, aucune alerte ne doit être égarée. SOAP implémente nativement WS-ReliableMessaging, assurant des accusés de réception au niveau transport. Une alternative REST aurait nécessité le développement sur mesure de mécanismes de retry et de webhooks pour atteindre le même niveau de robustesse.

Critère 2 — Formalisme contractuel strict (WSDL/XSD). Le système de maintenance exige un typage fort pour valider la localisation (Bâtiment/Zone) et la nature de la panne. Le contrat WSDL impose une validation par schéma XSD à la compilation et à l'exécution. Avec REST/JSON, un changement non

documenté ou une clé mal orthographiée côté partenaire provoquerait des pannes silencieuses en production sans rejet immédiat de la requête.

Critère 3 — Interopérabilité avec un système legacy. La GMAO du prestataire n'expose nativement qu'une interface SOAP. Choisir SOAP permet de consommer directement leur service sans surcoût. Une approche full REST aurait imposé de développer et maintenir une passerelle de traduction intermédiaire, augmentant la complexité et les points de défaillance.

Pourquoi SOAP a été écarté pour Pennylane

Pennylane s'intègre naturellement via des API HTTP/JSON orientées SaaS. Dans ce contexte, la qualité d'intégration dépend d'un bon mapping métier, de l'idempotence et de la gestion des erreurs applicatives — pas d'un contrat SOAP lourd. REST permet de conserver une architecture plus simple, plus rapide à faire évoluer et mieux alignée avec notre API existante.

Comparaison REST vs SOAP sur le cas ThermoSense

Critère	API REST actuelle	Approche SOAP/SOA (GMAO)
Formalisme du contrat	OpenAPI 3.0 (YAML) — évolutif mais non contraignant légalement	WSDL + XSD typé — toute modification détectée immédiatement
Testabilité	Facile : curl, Postman, pipeline CI	Plus lourde : SoapUI ou WireMock-SOAP requis
Évolutivité	Ajout de champ sans breaking change avec versioning d'URL	Toute modification du WSDL casse les clients existants
Complexité d'intégration	Faible : JSON + HTTP standard	Élevée : XML/SOAP, enveloppes, namespaces, WS-Security
Gouvernance	Interne, à autodiscipliner	Imposée par contrat bipartite — WSDL est le contrat légal

Limites et incertitudes

L'intégration SOAP avec la GMAO est architecturale dans le cadre de ce projet : le contrat WSDL n'a pas été implémenté faute d'accès au système partenaire réel. En production, deux incertitudes subsistent : la capacité du partenaire à absorber une charge synchrone importante en cas d'alertes simultanées massives, et le choix de mécanisme de sécurité (WS-Security avec jeton ou mTLS) à arbitrer avec leur DSI.

Section D — Résilience mobile & IoT

Patterns implémentés

Deux mécanismes principaux ont été implémentés pour répondre aux contraintes de réseau mobile instable et de fiabilité des données IoT.

Pattern 1 — Idempotency Key

Déclencheur : header **Idempotency-Key** ou champ **idempotencyKey** dans le body de la requête.

Comportement : si la clé est trouvée en cache, la réponse précédente est rejouée avec le header **Idempotent-Replayed: true** sans réexécuter l'opération. Si la clé est absente, le traitement normal s'effectue puis la réponse est sauvegardée en cache.

Justification métier : un capteur IoT sur réseau instable peut renvoyer la même mesure plusieurs fois sans savoir si la première requête a abouti. Sans idempotence, la série temporelle se remplit de doublons, ce qui fausse les moyennes et peut déclencher des alertes en double. Ce mécanisme est particulièrement critique sur **POST /actuators/{id}/commands** : une double activation d'un chauffage ou d'une vanne a des conséquences physiques réelles.

Configuration retenue : TTL cache 24h, nettoyage automatique toutes les heures.

Pattern 2 — Mode dégradé avec file d'attente

Déclencheur : base de données inaccessible, ou variable d'environnement **SIMULATE_DB_DOWN=true**.

Comportement : la mesure est mise en file d'attente en mémoire (**measurementQueue**) et une réponse **202 Accepted** est retournée au lieu d'un **500**. La file est vidée automatiquement toutes les 5 secondes avec un maximum de 5 tentatives par entrée.

Justification métier : dans un contexte IoT, perdre des relevés de capteurs crée des trous dans les données historiques qui compromettent les analyses de consommation et les détections d'anomalies. Répondre **202** au lieu de **500** évite également que l'appareil IoT entre dans une boucle de retry agressive.

Pattern 3 — Rate limiting différencié

Trois limiteurs distincts selon la criticité des endpoints :

- Login : 8 échecs / 10 min / IP — protège contre la force brute sur l'authentification
- Refresh token : limiteur dédié — protège contre l'abus de renouvellement de session
- Écritures critiques : 60 req / 15 min / IP — protège contre la saturation des actionneurs et la pollution de la base

Résultats de simulation

Les mécanismes ont été testés sous trois scénarios de dégradation réseau simulés via Postman :

Métrique	Baseline	S1 — 3G faible (200–500ms, 5% perte)	S2 — Réseau instable (800–1500ms, 20% perte)	S3 — Passerelle saturée (2000–4000ms, 50% perte)
Taux de succès	100%	100%	80%	75%

p95 latence perçue	12 ms	499 ms	1484 ms	3207 ms
Retries / requête	0	0	0,2	0,25
Doublons observés	0	0	4	5

Analyse des résultats et limites

Les doublons résiduels observés en S2 et S3 (4 à 5) s'expliquent par une limite de l'environnement de test : Postman ne permet pas de bloquer réellement les requêtes HTTP au niveau réseau. La perte de paquets y est simulée logiciellement, sans impact sur la couche transport. Les mécanismes côté serveur (idempotency key, mode dégradé, flush automatique) sont fonctionnels et ont été validés unitairement.

Configuration retenue et justifications

Paramètre	Valeur	Justification
Timeout client S1	3000 ms	Réseau 3G dégradé : latence p95 mesurée à 506 ms, marge confortable
Timeout client S2	5000 ms	Réseau très instable : latence p95 à 1459 ms, 5s évite les faux timeouts
Timeout client S3	8000 ms	Passerelle saturée : latence p95 à 3496 ms, 8s est la limite acceptable côté UX
TTL idempotency cache	24h	Couvre la fenêtre de retry maximale d'un appareil IoT sur panne prolongée
Flush mode dégradé	5s / 5 tentatives	Équilibre entre réactivité de récupération et pression sur la base

Limites restantes et axes d'amélioration

Limites actuelles :

- Le cache d'idempotence et la file de mode dégradé sont stockés en mémoire : un redémarrage du serveur efface les deux. En production, ces états doivent être externalisés sur Redis.
- Pas de mécanisme de retry côté client documenté : la stratégie de retry (nombre de tentatives, backoff exponentiel, délai) est laissée à l'implémentation du client mobile, sans spécification dans le contrat.
- Absence de notifications push : l'architecture est exclusivement pull. Les clients doivent interroger l'API pour détecter les changements d'état ou les alertes.

Pour aller plus loin en production :

1. Externaliser le cache d'idempotence et la file de mode dégradé sur Redis pour garantir la persistance inter-redémarrage et permettre le déploiement multi-instances.
2. Documenter et standardiser la stratégie de retry côté client dans le contrat OpenAPI (headers `Retry-After`, `X-RateLimit-*` déjà présents côté serveur).
3. Ajouter un mécanisme de push (webhooks ou MQTT bridge) pour les alertes et changements d'état critiques, réduisant la charge de polling en contexte IoT.